

2조 SOL로몬 — 설치 방법

우송대 RAG 기반 학사 정보 질의응답 챗봇의 설치 및 실행 방법입니다.

외부 서버 없이 이 PC 한 대에서 전체 시스템이 독립 실행됩니다.

1. 파일 구조

```
AI_Chatbot_Assistant/
├── BackEnd/
│   ├── app/
│   │   ├── agents/
│   │   │   ├── rag/
│   │   │   ├── services/
│   │   │   ├── api/
│   │   └── models/
│   ├── scripts/
│   ├── llm/
│   └── Dockerfile
├── FrontEnd/
├── models/
├── deploy/
│   ├── docker-compose.yml
│   ├── .env.example
│   ├── data/
│   └── 설치_방법.md
└── docker-compose.yml
```

API 서버 (FastAPI · Python)
애플리케이션 코드
LangGraph 대화 흐름 · 주제 분류
문서 색인 · 검색 · 리랭킹
기능별 처리 (졸업요건 · 장학금 · 학사일정 · FAQ)
HTTP 엔드포인트
데이터베이스 테이블 정의
운영 스크립트 (데이터 반입 · 회귀 테스트)
★ 답변 생성 모델을 여기에 넣습니다 (3번 참고)
웹 화면 (React · Vite)
★ 리랭커 모델을 여기에 넣습니다 (3번 참고)
배포용 파일
← 실행에 사용
← 환경변수 예시
학사 데이터 (아래 참고)
이 문서
개발용 (공용 서버 접속 - 사용하지 않습니다)

`deploy/data/` 에 들어 있는 것

파일	내용
<code>seed.sql</code>	학과 · 건물 · 졸업요건 · 학사일정 · 장학금 · FAQ · 주제 분류 문장 (1,589행)
<code>qdrant_points.jsonl</code>	학사 문서 검색용 벡터 606개 (원문 + 임베딩)
<code>files/</code>	학사 서식 · 장학금 공고 첨부파일 131개

개인정보는 포함되어 있지 않습니다. 학생 계정, 성적, 대화 기록은 모두 제외했습니다.

2. 사전 준비

2-1. 하드웨어

- **NVIDIA GPU** (VRAM 8GB 이상 권장, RTX 3070 기준으로 검증)

GPU가 없는 경우는 8-5번을 참고하세요.

- 저장 공간 **약 45GB** 여유

항목	용량
백엔드 Docker 이미지 (CUDA + PyTorch + llama-cpp)	19.8 GB
빌드 캐시 (빌드 중에만 사용, 이후 정리 가능)	약 10 GB
모델 파일 (3번)	7.1 GB
임베딩 캐시 (첫 실행 시 자동 다운로드)	4.6 GB
Postgres · Qdrant · 프론트엔드 이미지	0.9 GB
학사 데이터 (<code>deploy/data</code>)	0.2 GB

- 네트워크: 최초 설치에 **약 20GB**를 내려받습니다. 유선 연결을 권장합니다.

2-2. 소프트웨어

① NVIDIA 그래픽 드라이버

<https://www.nvidia.com/download/index.aspx> 에서 최신 버전 설치

설치 확인:

```
nvidia-smi
```

GPU 이름과 CUDA 버전이 표시되면 정상입니다.

② Git · Git LFS

<https://git-scm.com/download/win> 에서 설치 (설치 중 Git LFS 항목을 켜 둡니다)

3-2의 리랭커 모델을 받을 때 필요합니다. 설치 후 한 번만 실행하세요.

```
git lfs install
```

③ Docker Desktop

<https://www.docker.com/products/docker-desktop> 에서 설치

설치 후 **Settings** → **Resources** → **WSL Integration** 을 켜고,

Settings → **Docker Engine** 에서 GPU 지원이 활성화되어 있는지 확인합니다.

컨테이너에서 GPU가 보이는지 확인:

```
docker run --rm --gpus all nvidia/cuda:12.8.1-base-ubuntu22.04 nvidia-smi
```

GPU 정보가 나오면 정상입니다. 오류가 나면 8번(문제 해결)을 참고하세요.

3. 모델 다운로드 (약 7GB)

용량이 커서 소스에 포함하지 않았습니다. 두 개를 직접 받아 지정된 위치에 넣습니다.

3-1. 답변 생성 모델 — Bllossom-8B (4.9GB)

다운로드

https://huggingface.co/MLP-KTLim/llama-3-Korean-Bllossom-8B-gguf-Q4_K_M

받을 파일

```
llama-3-Korean-Bllossom-8B-Q4_K_M.gguf
```

넣을 위치

```
AI_Chatbot_Assistant/BackEnd/llm/llama-3-Korean-Bllossom-8B-Q4_K_M.gguf
```

3-2. 리랭커 모델 — bge-reranker-v2-m3-ko (2.2GB)

다운로드

<https://huggingface.co/dragonkue/bge-reranker-v2-m3-ko>

폴더 전체를 받습니다. `git lfs` 가 있으면 아래가 편합니다.

```
cd AI_Chatbot_Assistant\models
git lfs install
git clone https://huggingface.co/dragonkue/bge-reranker-v2-m3-ko
```

넣을 위치

```
AI_Chatbot_Assistant/models/bge-reranker-v2-m3-ko/
```

3-3. 임베딩 모델 — BGE-M3 (2.3GB)

받지 않아도 됩니다. 첫 실행 시 자동으로 내려받아 Docker 볼륨에 저장됩니다.

최초 1회만 시간이 걸리고 이후에는 재사용됩니다.

4. 환경변수 설정

`deploy\.env.example` 을 `BackEnd\.env` 로 복사합니다.

```
cd AI_Chatbot_Assistant
copy deploy\.env.example BackEnd\.env
```

복사한 `BackEnd\.env` 를 열어 아래 한 줄만 채우면 됩니다.

```
SECRET_KEY=여기에_임의의_긴_문자열을_넣으세요
```

값 생성 (PowerShell에서 그대로 실행 — Python이 없어도 됩니다):

```
-join ((48..57)+(65..90)+(97..122) | Get-Random -Count 48 | % {[char]$_})
```

형식 제한은 없습니다. 아무 긴 문자열이어도 동작합니다.

나머지 값은 그대로 두면 동작합니다.

VRAM이 8GB보다 크면 `LLM_GPU_LAYERS=-1`, `LLM_N_CTX=8192` 로 올리면 응답이 빨라집니다.

선택 기능(날씨 · 지도 · 다국어)은 각각 API 키가 필요하며, 비워두면 해당 기능만 동작하지 않습니다.

5. 실행

5-1. 컨테이너 기동

```
cd AI_Chatbot_Assistant
docker compose -f deploy/docker-compose.yml up -d --build
```

최초 빌드에 20~40분 걸립니다. `llama-cpp-python` 을 CUDA 소스에서 빌드하기 때문입니다.

두 번째부터는 캐시가 있어 수 초 만에 뜹니다.

진행 상황 확인:

```
docker compose -f deploy/docker-compose.yml logs -f backend
```

`Application startup complete` 가 보이면 준비된 것입니다.

(임베딩 모델 최초 다운로드 때문에 첫 기동은 3~5분 더 걸립니다.)

5-2. 학사 데이터 반입

컨테이너가 뜬 뒤 **한 번만** 실행합니다.

```
docker compose -f deploy/docker-compose.yml exec backend python3 -m scripts.import_deploy_data
```

아래처럼 나오면 성공입니다.

1) 테이블 생성	→ 테이블 생성/확인 완료
2) 참조 데이터	→ 참조 데이터 1600/1600 문장 적용
3) 첨부파일	→ 첨부파일 131/131개 적재
4) Qdrant	→ 포인트 606개 적재 완료

이미 데이터가 있으면 건너웁니다. 처음부터 다시 넣으려면 `--force` 를 붙입니다.

6. 접속 및 계정 만들기

주소	용도
http://localhost:5173	학생 화면 (챗봇)
http://localhost:8000/docs	API 문서

6-1. 학생 계정

개인정보를 제외했기 때문에 **계정이 하나도 없습니다.**

<http://localhost:5173> 에서 회원가입하면 바로 사용할 수 있습니다.

6-2. 관리자 계정

먼저 위와 같이 회원가입한 뒤, 아래 명령으로 권한을 올립니다.

20250001 자리에 가입할 때 쓴 학번을 넣으세요.

```
docker compose -f deploy/docker-compose.yml exec postgres psql -U postgres -d school_chatbot -c "UPDATE student SET role='admin' WHERE student_no='20250001';"
```

다시 로그인하면 관리자 화면이 열립니다.

7. 동작 확인

챗봇에 아래를 입력해 보세요.

질문	확인 대상
휴학 어떻게 해?	문서 검색 · 리랭킹
학교 좀 쉬고 싶은데	비정형 표현 인식
컴퓨터공학과 알려줘	학과 정보 카드
학사일정 알려줘	달력 카드
국가장학금 알려줘 → 신청은?	대화 맥락 유지

전체 기능을 한 번에 점검하려면:

```
docker compose -f deploy/docker-compose.yml exec backend python3 -m scripts.regression_battery
```

8. 문제 해결

컨테이너에서 GPU가 안 보입니다

```
could not select device driver "nvidia" with capabilities: [[gpu]]
```

Docker Desktop을 재시작하고, WSL2 백엔드가 켜져 있는지 확인하세요.

그래도 안 되면 그래픽 드라이버를 최신으로 올리고 PC를 재부팅합니다.

모델 로딩에 실패합니다

```
Failed to load model from file: ./llm/...gguf
```

3-1의 파일명과 위치가 정확한지 확인하세요. 파일명이 한 글자라도 다르면 실패합니다.

포트가 이미 사용 중입니다

```
Bind for 0.0.0.0:5432 failed: port is already allocated
```

PostgreSQL이 이미 PC에 설치되어 있는 경우입니다.

`deploy/docker-compose.yml` 에서 `"5432:5432"` 를 `"5433:5432"` 로 바꾸세요.

(컨테이너끼리는 내부 이름으로 통신하므로 `.env` 는 고치지 않아도 됩니다.)

응답이 매우 느립니다

GPU 대신 CPU로 돌고 있을 가능성이 높습니다. 아래로 확인하세요.

```
docker compose -f deploy/docker-compose.yml exec backend nvidia-smi
```

처음부터 다시 하고 싶습니다

```
docker compose -f deploy/docker-compose.yml down -v
```

`-v` 는 데이터베이스와 Qdrant 내용까지 모두 지웁니다. 이후 5번부터 다시 진행하세요.

8-5. NVIDIA GPU가 없는 PC에서 실행하려면

이 방법은 검증하지 않았습니다. GPU가 있는 환경을 강하게 권장합니다.

GPU가 없으면 컨테이너가 아예 뜨지 않습니다. 아래 두 가지를 바꿔야 합니다.

① `deploy/docker-compose.yml` 의 `backend` 에서 GPU 예약 블록을 지웁니다

```
deploy:                                # ← 이 5줄을
  resources:                            #   통째로 삭제
    reservations:
      devices:
        - driver: nvidia
```

② `BackEnd/.env` 를 CPU로 바꿉니다

```
DEVICE=cpu
EMBEDDING_DEVICE=cpu
RERANKER_DEVICE=cpu
```

이 상태에서 검색과 리랭킹은 느려도 동작하지만, **로컬 8B 모델의 답변 생성은 한 건에 수 분이 걸려 사실상 사용하기 어렵습니다.**

실용적인 대안은 답변 생성만 클라우드로 돌리는 것입니다. Vertex AI 서비스계정 키(`gcp-sa.json`)를 별도로 준비한 뒤 아래처럼 바꿉니다.

```
LLM_PROVIDER=vertex
GCP_PROJECT_ID=[프로젝트 ID]
```

키 파일은 보안상 배포본에 포함하지 않았습니다. 필요하다면 별도로 문의하세요.

9. 시스템 구성 요약

